INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

CS5903 – Distributed AI Training

AdaptEngine

Project 3

Adaptive Learning, Diagnostic Assessment
and Personalized Path-Planning Engine

Weeks 4–5 Progress Report

Vuppula Hemanth Reddy

AI23BTECH11033

Group 15

Github Repository

`https://github.com/vhemanth06/AdaptEngine`

Contents

1 Overview of Weeks 4 and 5 **1**

2 Work Completed in Week 4: Planner and Difficulty Policy **1**

2.1 How the planner works(M3) 1

2.2 Exact checker and honest answers 2

2.3 Difficulty policy (M4) 3

2.4 Key decisions 3

2.5 Larger test curriculum 3

2.6 Scenarios checked 4

3 Work Completed in Week 5: Review, Remediation and Replanning **4**

3.1 Spaced review (M5) 4

3.2 Remediation (M5) 5

3.3 One shared daily budget 5

3.4 Replanning after events (M6) 5

3.5 Decision traces (M7) 6

3.6 Key decisions 6

3.7 Scenarios checked 6

4 Fixes Following the Week 3 Mentor Review **7**

5 End-to-End Demonstration **8**

6 Evaluation Against Baselines (M8, Part 1) **8**

7 Current Progress **9**

7.1 Milestone Progress 9

7.2 Acceptance Test Register 10

8 Future Work (Week 6 Plan) **11**

1 Overview of Weeks 4 and 5

Before, AdaptEngine could load a reviewed curriculum, estimate a learner's mastery with a short diagnostic, and record events safely. It could not yet turn that knowledge into a study plan. In Week 4 and Week 5, the project filled this gap and also repaired the problems found in the Week 3 mentor review.

- **Week 4 (M3, M4).** A time-budgeted planner that builds a day-by-day schedule without breaking prerequisites or the daily budget, an exact checker for tight cases, a clear "cannot fit" answer with corrective options, and a rule for choosing the difficulty of the next exercise.
- **Week 5 (M5, M6, M7).** Spaced review based on forgetting, remediation after a failed exercise, replanning after all four required events, and decision traces that explain every choice.
- **Mentor-review fixes.** Every item from the Week 3 review was addressed, including the unsafe save path, the pyBKT integration, ignored settings, loose file loading, and mismatched models in the diagnostic.

Table 1: Summary of the two weeks.

Area	What was added	Main evidence
Week 4	Planner, feasibility bounds, exact checker, difficulty policy, 8-concept fixture	TC3.1–TC3.6, TC4.2, TC2.3
Week 5	Forgetting model, review and remediation, four-event replanner, traces	TC5.1–TC5.3, TC6.1–TC6.4, TC7.1
Review fixes	Atomic save, real pyBKT, strict loading, one model, session IDs	16 dedicated regression tests
Demonstration	Diagnostic to plan to update to replan	One script and its test
Evaluation	25 seeded learners, three diagnostic arms, two planner arms	TC8.1 and saved raw results

2 Work Completed in Week 4: Planner and Difficulty Policy

2.1 How the planner works(M3)

The planner answers one question: given what this learner already knows, what is the best valid way to spend their minutes before the deadline? It works in four steps.

1. **Find the work that is needed.** Starting from the goal, it walks back through the prerequisite graph and drops every concept the learner has already mastered with evidence. Each remaining concept gets a projected to-do list of lessons and exercises, estimated by assuming correct responses and simulating how mastery would rise. A beginner gets a longer list, while an advanced learner gets a shorter one.

2. **Filter infeasible goals early.** Before simulating daily schedules, cheap analytical checks test whether the goal is mathematically possible: is any single activity longer than the daily budget, does total work exceed the total available minutes, or is the longest dependent chain (critical path) too long? If any check fails, the planner halts immediately with concrete shortfall options.

3. **Pack each day greedily.** The planner fills one day at a time, keeping hard validity constraints separate from preference scoring. An activity is eligible only if its prerequisites are finished and it fits within the day's remaining minutes. Among eligible activities, heuristic scores decide the order (without ever violating constraints). If the highest-scoring activity does not fit the remaining minutes, it is deferred and the next highest-scoring fitting activity is scheduled, because activities cannot be split.

4. **Resolve tight schedules with an exact search.** If the fast bounds pass but the greedy plan exceeds the deadline due to packing fragmentation, an exact branch-and-bound bin-packing oracle searches for an optimal sequence respecting precedence constraints before declaring the deadline unreachable.

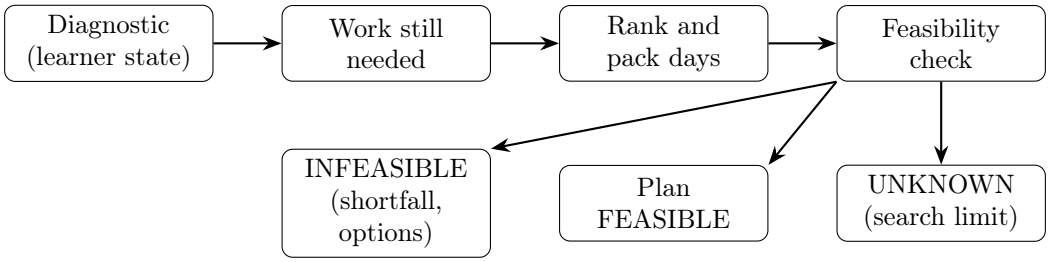


Figure 1: Planner flow. A plan is returned only when it is valid; otherwise the answer is INFEASIBLE with reasons, or UNKNOWN.

2.2 Exact checker and honest answers

The greedy scheduler is fast, but because it packs one day at a time using local heuristic scores, it can fragment daily time budgets and miss a tight, non-obvious schedule. Conversely, simple continuous checks ($\sum d_i \leq D \cdot B$) only sum total minutes and cannot capture discrete boundaries, because activities cannot be split across days.

When cheap checks pass but greedy packing exceeds the deadline, the planner does not immediately declare failure. Instead, it invokes an **exact search**—a branch-and-bound bin-packing oracle that systematically explores all topologically valid assignments of activities to daily bins.

This exact search serves three critical roles:

- **Rescuing valid plans:** If a feasible packing exists that the greedy heuristic missed due to local ranking choices, the exact search finds it and returns `FEASIBLE`.
- **Certifying genuine infeasibility:** Consider an item set of 8, 8, and 4 minutes with a 10-minute daily budget over two days. The 20-minute total perfectly matches two full days, so cheap checks pass. However, neither 8-minute activity leaves enough remaining capacity (2 minutes) for the 4-minute activity. Greedy packing takes three days, and the exact search exhaustively verifies that no two-day assignment exists, definitively certifying the plan as `INFEASIBLE`.
- **Bounded computation and honest uncertainty:** Because precedence-constrained bin-packing is an NP-hard combinatorial problem, exploring every permutation for large plans could cause exponential slowdowns. To preserve strict determinism without non-

deterministic wall-clock timeouts, the search is capped by an integer node expansion limit. If the limit is reached without finding a solution, the engine halts with `UNKNOWN`—guaranteeing it never outputs a false `INFEASIBLE`.

Whenever a goal cannot fit within the deadline (whether proven `INFEASIBLE` or flagged `UNKNOWN`), the planner does not merely fail. It calculates the exact shortfall in days and minutes and generates concrete, actionable options for the learner, such as the exact deadline extension or the minimum viable daily budget required to succeed.

2.3 Difficulty policy (M4)

To maintain learner engagement within the Zone of Proximal Development, the planner selects exercises targeting a predicted success rate of $P(\text{correct}) \approx 75\%$. Among available exercises for a concept, the engine minimizes $|P_c - 0.75|$. Ties are resolved deterministically by choosing the item closest to median difficulty ($|d - 0.50|$), then by lexicographical activity identifier.

For a novice learner with low latent mastery, the 75% success ceiling may be unreachable even on simple material; in this case, the closest possible match is mathematically the easiest available exercise. (Instructional lessons carry neutral difficulty and are always scheduled before exercises.)

2.4 Key decisions

The decisions below were written down and frozen before the tests were finalised.

Table 2: Week 4 design decisions.

ID	Topic	Decision
D11	Multi-day work	Later days are planned by assuming scheduled exercises are answered correctly.
D12	Same-day unlock	A concept finished early in a day may have dependent activities later the same day.
D14	Lessons and difficulty	Lessons have neutral difficulty and are queued as soon as a concept is eligible.
D15	Packing rule	An activity that does not fit is skipped; smaller ones are still tried.
D16	Order inside a concept	Lessons always come before exercises.
D17	Search limit	The exact search stops at a fixed node limit and then reports UNKNOWN.
D19	Feasibility bounds	INFEASIBLE is proven when total work or the longest dependency chain exceeds deadline times budget.

2.5 Larger test curriculum

The four-concept curriculum is too small to show planning behaviour well, so an eight-concept curriculum was added with varied prerequisites, 16 lessons and 32 exercises with difficulties between 0.15 and 0.85. Both curricula go through the same review gate.

2.6 Scenarios checked

Table 3: Week 4 acceptance scenarios and results.

ID	Scenario	Result
TC2.3	Two learners with different prior mastery, same goal and budget	Different plans, each valid; mastered concepts omitted with the evidence recorded.
TC3.1	Unmet prerequisite	Nothing is scheduled before its prerequisites are evidence-mastered.
TC3.2	Budgets of 5, 10, 15 and 30 minutes	No day ever exceeds its budget.
TC3.3	Same learner at 5 versus 30 minutes	5 minutes stretches the work over many more days.
TC3.4	Goal C4 in one day at 5 minutes	INFEASIBLE with the exact minute shortfall and options.
TC3.5	8 + 8 + 4 minutes, 10-minute budget, two days	Cheap checks pass; exact search reports INFEASIBLE.
TC3.6	Search stopped by the node limit	UNKNOWN, no crash and no false INFEASIBLE.
TC4.2	Choose the next exercise for a given mastery	Predicted success nearest 75%; ties follow the stated rule.

3 Work Completed in Week 5: Review, Remediation and Replanning

3.1 Spaced review (M5)

Learners forget over time. The engine models the chance of remembering a concept using an exponential decay curve:

$$r(t) = 2^{-\Delta t/h},$$

where Δt is the number of days since the concept was last practised and h is its half-life (30 days by default). A concept is excluded from decay until it is practised for the first time.

When recall falls below 0.80, a review becomes due. If multiple concepts need review on the same day, an urgency score decides which one comes first:

$$\text{Urgency} = (1 - r)\,(1 + \text{descendant_count}),$$

where descendant_count is the number of downstream concepts that depend on it in the prerequisite graph. This formula balances two things: how much has been forgotten $(1 - r)$ and how important the concept is as a foundation. The +1 ensures that even an end-topic with zero dependencies still gets reviewed when forgotten.

When filling each day's study budget, the planner gives reviews priority over new lessons:

$$\text{Remediation} \longrightarrow \text{Spaced Reviews} \longrightarrow \text{New Lessons}.$$

This ensures students refresh fading fundamentals before taking on new material. Review exercises follow the same 75% target success rule, and repeating an earlier exercise is permitted for review.

3.2 Remediation (M5)

When a learner struggles, the engine steps in with corrective instruction. If a wrong answer leaves the learner's mastery below 0.40, the concept is flagged for remediation (`needs_remediation = True`).

Crucially, this flag is triggered by an actual failed attempt, rather than low mastery alone. This distinction prevents brand-new, untouched concepts (which naturally start with low initial mastery) from being mistaken for failed topics.

Once flagged, the engine responds in three steps:

1. **Top priority (Tier 0):** In the next plan, the remedial lesson is scheduled ahead of all spaced reviews and new lessons, consuming the first minutes of the daily budget.
2. **Restoring fundamentals:** The concept's instructional lesson is re-issued to rebuild understanding before any further exercises are given.
3. **Clearing the flag:** Completing the repeat lesson raises the learner's prior belief (by $T_{\text{lesson}} = 0.20$), clears the remediation flag, and safely resumes normal practice.

3.3 One shared daily budget

New lessons, review and remediation compete for the same few minutes. The priority order is fixed.

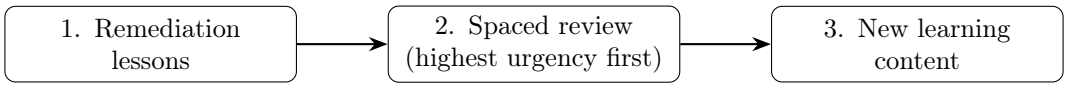


Figure 2: Daily priority order under a shared time budget.

3.4 Replanning after events (M6)

Replanning is a fresh planner run starting from the current day. It takes the learner's updated state, the record of completed work, and the original settings, with no hidden state.

The effective deadline and daily budget are calculated by applying all recorded events to the initial configuration. Any completed activity is permanently marked as finished (`REPLAN_PRESERVED_COMPLETED`), guaranteeing that past work is never lost and never repeated.

Table 4: The four dynamic disruption events and engine responses.

Event	What the engine does
Missed day (`MISSED`)	Records zero minutes for that date and starts the new plan from the next day (day + 1). Activities scheduled for the missed day return to the queue.
Failed assessment (`FAILED_ASSESSMENT`)	Updates mastery downward, flags the concept for remediation ($p < 0.40$), and repacks remaining days with a repeat lesson at Tier 0.
Deadline changed (`DEADLINE_CHANGED`)	Replans over the new deadline, or proves the goal `INFEASIBLE` with shortfall options if the deadline shrank too far.

Event	What the engine does
Budget changed (`BUDGET_CHANGED`)	Adjusts daily capacity to the new minute limit and repacks all remaining work.

Working ahead: Work finished ahead of schedule is also credited in the log, so subsequent activities automatically move forward into earlier days.

3.5 Decision traces (M7)

Every plan carries a machine-readable trace. Each selected activity records its reason and score components; each omitted concept records the evidence that justified omitting it (for example its mastery and number of responses); each activity that did not fit a day is marked as deferred. Traces contain only plain values, so they can be saved as JSON and printed with the `-json` option.

3.6 Key decisions

Table 5: Week 5 design decisions.

ID	Topic	Decision
D20, D21	Remediation	Triggered by a failed answer leaving mastery below 0.40; a repeat lesson is queued first; flag cleared after the lesson.
D22	Unpractised concepts	Not review candidates until practised.
D23	Forgetting model	Fixed half-life decay, as in the project decisions.
D24, D25	Review trigger and urgency	Due below 0.80 recall; urgency weighted by dependants.
D26	Priority order	Remediation, then review, then new content.
D27	Review exercises	Chosen by the 75% rule; repeats allowed.
D28–D30	Events and replanning	Deadline and budget derived from the event log; a missed day replans from the next day.

3.7 Scenarios checked

Table 6: Week 5 acceptance scenarios and results.

ID	Scenario	Result
TC5.1	Concept already mastered	Left out of new learning; the trace cites the evidence.
TC5.2	Failed exercise leaves mastery below 0.40	Remediation lesson inserted next day; flag set, then cleared after the lesson.

ID	Scenario	Result
TC5.3	High review urgency with spare time	Review is scheduled before lower-urgency new content.
TC6.1	Learner misses a day	New valid plan from the next day; nothing lost or duplicated.
TC6.2	Failed assessment	State updated; revised plan with remediation.
TC6.3	Deadline changed	Valid revised plan or an INFEASIBLE proof with shortfall.
TC6.4	Daily budget changed	Revised plan fits the new budget.
TC7.1	Plan with selected and omitted activities	Every decision traceable to state, constraints and score parts; trace saves as JSON.

4 Fixes Following the Week 3 Mentor Review

The Week 3 review accepted project progress but identified several claims that were not fully realized. Each point was addressed and verified with regression tests:

- **Atomic persistence (FB006):** Previously, saving learner state and logs in separate files risked corruption if an interruption occurred between writes. The engine now writes state, log, and context together to a temporary file and swaps them in using a single atomic filesystem operation.
- **Authentic pyBKT integration (FB001):** The earlier adapter calculated its own arithmetic rather than calling the actual pyBKT library. The engine now genuinely runs pinned upstream pyBKT (`pybkt==1.4.3`), verifies parity to within 10^{-9}, and fails tests if the library is missing.
- **Config-driven mastery settings (FB002):** Mastery checks previously bypassed configuration settings and relied on hardcoded constants (0.85 and 2). A single canonical function now reads configured thresholds everywhere, enforces positive integer caps, and rejects invalid difficulties.
- **Strict file loading (W2-FB005):** Loose parsing previously converted invalid input strings (such as the text `"false"`) into boolean `True`. Strict deserialization now validates all inputs and rejects malformed types, negative uncertainty, or invalid half-lives.
- **Unified observation model (FB002):** Diagnostic question selection previously calculated expected information gain using internal formulas rather than the active model. Both question selection and posterior updating now share the exact same `MasteryModel` instance.
- **Session-scoped attempt IDs (FB002):** Distinct diagnostic runs previously generated colliding attempt identifiers. Attempt IDs now include a unique session identifier, making replays idempotent while allowing distinct sessions to accumulate evidence.
- **Lessons versus evidence contract (D31):** The boundary between instruction and evidence was formalized: lessons raise belief but do not count as assessment responses. Evidence-mastery strictly requires both passing the mastery threshold ($p \geq \theta$) and completing enough real exercise attempts.

- **Unknown goals and CLI exit codes:** Unknown goal concept names were previously ignored rather than rejected. The engine now raises a clear `UnknownConceptError`, and the command line returns standardized exit codes: 0 for `FEASIBLE`, 1 for `INFEASIBLE`, and 3 for `UNKNOWN`.

5 End-to-End Demonstration

The review asked for one runnable example of the full pipeline. A single script now shows it on the curriculum, and a test checks its key facts.

1. **Two learners.** A novice (initial mastery 0.20) needs a short diagnostic and receives a plan of about five days. An advanced learner (initial mastery 0.70) is shown to know C1 and C2 and receives a plan of two days aimed at C3 and C4.
2. **A recorded answer.** One exercise response is saved to disk through the atomic snapshot, and a revised plan follows.
3. **Four budgets.** The same learner is planned at 5, 10, 15 and 30 minutes per day.
4. **A request that cannot fit.** Reaching C4 in one day at five minutes needs 26 minutes of work; the engine reports INFEASIBLE with a 21-minute shortfall and options such as a longer deadline.
5. **All four events and a review.** A missed day, a failed assessment (which inserts a remediation lesson), a deadline shortened from 10 to 6 days, and a budget raised from 15 to 20 minutes each produce a valid revised plan. Memory decay below 0.80 adds a review exercise.
6. **Explanations.** A readable justification is printed for every scheduled activity.

6 Evaluation Against Baselines (M8, Part 1)

To evaluate the engine objectively, we benchmarked it against standard baselines across 25 seeded synthetic learners. To prevent bias, learner responses are generated using an independent two-parameter logistic item-response theory (2PL-IRT) model rather than the engine's internal BKT rules. Hidden true masteries are never exposed to the algorithms, and all arms are tested on the exact same paired learners.

Experimental arms:

- **Diagnostic arms:** Adaptive selection via Expected Information Gain (EIG) with entropy stopping and caps; a fixed round-robin order; and uniform random selection. The fixed and random baselines always consume their full 8-item budget.
- **Planner arms:** AdaptEngine with dynamic event replanning versus a static greedy planner that follows its initial schedule without adapting when disruptions occur.

Table 7: Diagnostic comparison across 25 seeded learners.

Diagnostic Arm	Estimation Error (MAE)	Mean Items Used	Item Cost
Adaptive (AdaptEngine)	0.349	2.72	34%

Diagnostic Arm	Estimation Error (MAE)	Mean Items Used	Item Cost
Fixed order	0.264	8.00	100%
Random selection	0.279	8.00	100%

Analysis of results. The evaluation highlights two core findings:

1. **Diagnostic efficiency trade-off:** The adaptive diagnostic slashes assessment burden by 66%, requiring an average of only 2.72 questions compared to the full 8.00 questions required by fixed and random approaches. However, its mean estimation error is higher (0.349 versus 0.264 and 0.279). Because the adaptive arm halts early once entropy bounds are met, it trades fine-grained accuracy for questionnaire brevity. The expectation that error would strictly match fixed and random baselines was therefore not met on this run; this trade-off is recorded honestly for further tuning in Week 6.
2. **Planner resilience under disruption:** Both the adaptive and static planners maintained 100% constraint safety, with zero prerequisite or daily budget violations. However, when faced with an unexpected missed study day, AdaptEngine's dynamic replanning salvaged the schedule and achieved a **25.1% relative increase** in final true mastery (0.637 versus 0.509) by repacking missed material instead of leaving it behind.

7 Current Progress

7.1 Milestone Progress

Table 8: Milestone status at the end of Week 5.

Milestone	Status	Note
M1	Done	Curriculum, review gate and learner state; strict file loading added.
M2	Done	Diagnostic uses the same model for choosing and updating; session-scoped IDs. Error comparison with fixed and random is open (see Section 6).
M3	Done	Planner, feasibility bounds, exact checker, shortfall options.
M4	Done	Mastery update, real pyBKT adapter and next-exercise difficulty policy.
M5	Done	Forgetting model, review urgency, remediation.
M6	Done	Replanning after all four events and working ahead.
M7	Done	Machine-readable decision traces and readable explanations.
M8	Partial	Paired baseline run and TC8.1 done; TC8.2 (frontier replanning) planned for Week 6.

7.2 Acceptance Test Register

The register has 23 core test cases. Each ID states its milestone and number (TC4.2 is the second test of M4).

Table 9: Acceptance test register with current status.

ID	Scenario	Expected result	Status
M1 – Curriculum and learner state			
TC1.1	Cycle in the prerequisite graph	Rejected with a typed error	Done (W2)
TC1.2	Valid and invalid content, state, event and configuration; unreviewed content	Valid loads; invalid rejected before any change	Done (W3)
M2 – Adaptive diagnostic			
TC2.1	Diagnostic with synthetic learners	Stops at threshold or cap; error no worse than fixed and random	Cap and stop done; error comparison not met in W5 run
TC2.2	Concepts with no observations	Prior kept, UNASSESSED, treated as unmastered	Done (W3)
TC2.3	Two learners with different priors	Different valid plans	Done (W4)
M3 – Planning and feasibility			
TC3.1	Unmet prerequisite	Locked concept never scheduled early	Done (W4)
TC3.2	Fixed daily budget	No day exceeds the budget	Done (W4)
TC3.3	5 versus 30 minutes	Clearly different pacing	Done (W4)
TC3.4	Goal cannot fit deadline	INFEASIBLE with shortfall and options	Done (W4)
TC3.5	$8 + 8 + 4$ packing, 10-minute budget	Bounds pass, exact search INFEASIBLE	Done (W4)
TC3.6	Search stopped by limit	UNKNOWN, never false INFEASIBLE	Done (W4)
M4 – Mastery update and difficulty			
TC4.1	Wrong answer at $p = 0.5$, $d = 0.5$; adapter versus library	0.1707 before and 0.2122 after the transition; adapter matches the library	Done (W3, real library from W5)
TC4.2	Next exercise for a given mastery	Success nearest the target; ties by rule	Done (W4)
M5 – Review and remediation			
TC5.1	Concept already mastered	Omitted; trace cites evidence	Done (W5)

continued on next page

ID	Scenario	Expected result	Status
TC5.2	Failed exercise leaves $p < 0.40$	Remediation lesson inserted; flag updated	Done (W5)
TC5.3	High review urgency, spare time	Review before lower-urgency new content	Done (W5)
M6 – Replanning			
TC6.1	Missed day	New valid plan; nothing lost or duplicated	Done (W5)
TC6.2	Failed-assessment event	Updated state; plan with remediation	Done (W5)
TC6.3	Deadline changed	Valid plan or INFEASIBLE proof	Done (W5)
TC6.4	Budget changed	Plan satisfies the new budget	Done (W5)
M7 – Decision traces			
TC7.1	Plan with selected and omitted activities	Every decision traceable	Done (W5)
M8 – Evaluation			
TC8.1	Run the harness twice with the same seeds	Identical raw results	Done (W5)
TC8.2	Frontier replanning versus full recomputation	Identical decisions; latency recorded	Planned (W6)

8 Future Work (Week 6 Plan)

With Milestones M1 through M7 complete, Week 6 focuses on completing the experimental evaluation of Milestone M8:

- **Deliver Milestone TC8.2 (Replanning vs. Recomputation):** Benchmark frontier event replanning against full recomputation from scratch, measuring execution runtime, step latency, and schedule stability under disruptions.
- **Investigate and Tune Diagnostic Accuracy:** Address the trade-off observed in the initial benchmark (where saving 66% of questions yielded slightly higher error). Experiment with adjusted entropy stopping thresholds (H_{stop}) and item caps on paired learners to improve accuracy without losing questionnaire brevity.
- **Scale Synthetic Evaluation:** Re-run the benchmark across larger, more diverse learner cohorts (e.g., rapid learners, forgetful learners) and test engine sensitivity across key thresholds (θ, $k_{\text{responses}}$, and retention half-life h).
- **Prepare Week 6 Progress Deliverables:** Package the experimental receipts, update benchmark figures, and document findings for the milestone review.